Chess

Introduction

This assignment goes through Inheritance in Python, so you learn about Object Oriented Programming. But we also want to explore Git possibilities by using this application as a demo. By using Decorators, List and Dict comprehensions and Generators, combined with the knowledge of Inheritance, you will be able to create a simple project.

- Introduction
- Objective
  - Knowledge
  - Skills
- Assignment explanation
- Getting started
  - Git checkpoint
- Setting up the pieces
  - Test out the Inheritance
  - Git checkpoint
  - Branch workflow for this assignment
- Testing the pieces
  - Implementing the first benefit of the abstractmethods.
- Setting up the board
  - Creating some more helper functions into the board
    - `setup_board()`
    - `print_board()`
  - Git checkpoint
    - `find_piece(symbol: str, identifier: int, color: str)`
  - Other scripts
    - `kill_piece(square)`
- Implementing the movements on the board
  - Define a BoardMovement class
  - Forward
  - Backward, Left, Right
  - More dynamic movements
  - Combination movements (Diagonally)
  - Knight movements

Using the movements
Implementing the movement onto the board
Implementing the movement onto the board, Take 2!
Git branching tip
Saving the board states
Loading in the states of the board using a generator
Expanding on the functionalities
Final Git checklist
Pull Request checklist (recommended)

Objective

Knowledge

- Python
  - Decorators
  - Generators
  - Dictionaries
  - Lists
  - Comprehensions
- OOP
- Git & GitHub

Skills

- Working with Python to create an OOP application
- Creating Python code based on requirements from a Chess game
- Python Decorators
- Staticmethods, classmethods, properties and methods
- Poetry or conda as a package and virtual environment manager

Assignment explanation

The assignment is about creating a simple game. We will create a chess game, but only a textual representation in the dictionary states. Later on, we can re-use this project in some of our other topics.

The Chess game will be far from finished, but it will give you a good idea of Inheritance in Python, with some interesting use cases of abstract methods, static methods and fun Python syntaxes. You could work it out as far as you think is necessary to understand all the concepts.

For those who want to experiment with the Chess games, you can use the `Pygame` library to create a graphical representation of the game. Numerous libraries exist that can be used to help with playing Chess.

Getting started

At first, create a new directory called `chess` with a `main.py` file, which will be the entrypoint we use to run and start the project. We won't be doing much in there yet.

Git checkpoint

Initialize your repository early and make your first commit before coding:

```
git init
git add .
git commit -m "chore: initialize chess assignment structur
e"
```

If you are using GitHub, create the remote repository and push:

```
git remote add origin <your-repo-url>
git branch -M main
git push -u origin main
```

Setting up the pieces

In the `chess` directory, create a `pieces.py` file.

The `pieces.py` file will contain a **Base** class which is called `BaseChessPiece` and defines the functions a chess piece should be able to do:

- Move
- Die

It does not contain an implementation of these functions yet, but it will be used as a base class for the other pieces. To define methods that can be overridden

with a different implementation in the **Parent** classes, we define them as a `@abstractmethod` by adding that decorator above the methodname. This decorative method is imported from the `abc` library. The `BaseChessPiece` should also implement the `ABC` class which is from the same library.

Only the `die()` function should toggle the `is_alive` property to false.

As we will be using inheritance, we will be able to use the functions of the `BaseChessPiece` class in the other pieces. We will also be able to use the properties of the `BaseChessPiece` class in the other pieces. The properties that are used for all the pieces will be `color`, `name`, `symbol`, `identifier` and `position`. An `is_alive` property is also very useful to use, The `symbol` property will be a single character, which will be used to represent the piece on the board. Use these symbols:

- Pawn: '-'
- Rook: 'R'
- Bishop: 'B'
- Knight: 'N'
- King: 'K'
- Queen: 'Q'

The `position` property will be the name of the position on the board. This could be `A3` or `H6` for example. This will be calculated when the board is attached in the next steps of this project. For now, hardcode this to 'None'.

the `color` property can only be `BLACK` or `WHITE`. You could use an Enum for that, if you like to. I will just hardcode it as a string (for now).

When we want to print our BaseChessPiece, the internal `__str__` method of a class object gets called. Use this method to return the `color`, `name` and `identifier` in one string: E.g.: `BLACK Rook 1` or `WHITE Queen 1`. Copy the same implementation in the `__repr__` method, which is how the BaseChessPiece will get **repr**esented when they get returned.

Test out the Inheritance

Test out your inheritance by creating a `Pawn` class that inherits from the `BaseChessPiece` and implements the `move()` function. Currently hardcode it to return a string `print("Pawn moves forward 1 position")`. Later on, we will implement the actual movement.

Remember that Python needs the `self` property to be added into each function. This is the way Python knows which object is calling the function. In the `__init__()` function, we can call the constructor of the parent class by using `super().__init__()`. By passing the `color` and `identifier` property we get from the `Pawn`s constructor, we can set the properties on our base class. This way, we can use the properties of the `BaseChessPiece` class in the `Pawn` class.

Continue by setting up a `Rook` class, which can move in a straight line, and a `Bishop` class, which can move diagonally. The `Queen` class can move in both directions of the Bishop and Rook. The `King` class can move in all directions, but only one position at a time. It can also kill any piece that is in its way. The `Knight` class can move in an L shape, but can only kill pieces that are in its way.

(Tip: Use GitHub Copilot to autocomplete your code, create useful comments for it to understand your code better)

Git checkpoint

Commit after implementing each piece type or small group of piece types. Example:

```
git add chess/pieces.py
git commit -m "feat: add pawn, rook and bishop movement stu
bs"
```

Branch workflow for this assignment

Use a dedicated branch per topic, instead of mixing everything into `main`.

```
git checkout main
git pull origin main
git checkout -b feature/chess-pieces
```

When you finish that topic:

```
git add .
git commit -m "feat: implement chess pieces and base moveme
nt stubs"
git push -u origin feature/chess-pieces
```

Repeat this approach for board logic and movement logic, for example:

- `feature/board-setup`
- `feature/movement-rules`
- `feature/decorators-and-state`

Testing the pieces

In a new `main.py` file create a `Pawn` object. this object should be imported from the `pieces.py` file using `from pieces import Pawn`. Initialize the `Pawn` object with the `color` on `BLACK` and the `identifier` on `1`. Call the `move()` function on it. This should print what you defined it to print.

Implementing the first benefit of the abstractmethods.

Currently, you have defined a `print(movement)` statement in each of the ChessPieces you defined. The Pawn had a `Pawn moves forward 1 position` string printed. Notice the repeated `print()` statement? We can solve that! Later on, we will be doing many more calculations other than printing this useless string.

As we are working with abstractmethods, we can call that method from the **Child** (I.e.: The Pawn) `move()` function. In this function, define the string as a seperate variable called `movement`. Execute the **Parent**'s `move()` function using `super().move(movement)`.

Make sure to update the `BaseChessPiece.move()` function to receive this `movement` parameter and to print it to the terminal.

Setting up the board

All of our Chess Pieces should be added to some kind of Chess Board. As we are not using a GUI (Graphical User Interface), this board will just be a textual representation.

I chose to work with a dictionary where the keys are the position names of the square positions.

In the `chess_01` directory, create a `board.py` file. In the file, we will create a `Board()` class. In the constructor (`__init__()`) method of this function, we use a dict comprehension to create a dictionary of `squares` (use `squares` as the property name). The keys are the square names and the values are None (for now). You can use the `range(ord('a'), ord('i'))` to create a range for the alphabetical characters and `range(1, 9)` to create a range for the numbers.

Those represent the rows and columns of the chess board. The alphabetical characters are the columns, whereas the numerical characters are the rows. The result will look like this:

```
{'a1': None, 'a2': None, 'a3': None, 'a4': None, 'a5': Non
e, 'a6': None, 'a7': None, 'a8': None, 'b1': None, 'b2': No
ne, 'b3': None, 'b4': None, 'b5': None, 'b6': None, 'b7': N
one, 'b8': None, 'c1': None, 'c2': None, 'c3': None, 'c4':
None, 'c5': None, 'c6': None, 'c7': None, 'c8': None, 'd1':
None, 'd2': None, 'd3': None, 'd4': None, 'd5': None, 'd6':
None, 'd7': None, 'd8': None, 'e1': None, 'e2': None, 'e3':
None, 'e4': None, 'e5': None, 'e6': None, 'e7': None, 'e8':
None, 'f1': None, 'f2': None, 'f3': None, 'f4': None, 'f5':
None, 'f6': None, 'f7': None, 'f8': None, 'g1': None, 'g2':
None, 'g3': None, 'g4': None, 'g5': None, 'g6': None, 'g7':
None, 'g8': None, 'h1': None, 'h2': None, 'h3': None, 'h4':
None, 'h5': None, 'h6': None, 'h7': None, 'h8': None}
```

Creating some more helper functions into the board

A few functions we can use in the `Board` class are:

`setup_board()`

`setup_board()` will add all the pieces to the board using the `squares` property. This will be done using the piece classes we created earlier on. Take a look at this example:

```
from pieces import Rook, Knight
self.squares['a1'] = Rook('BLACK', 1) # This defines the fi
rst Rook of the BLACK team.
self.squares['b1'] = Knight('BLACK', 1) # This defines the
first Knight of the BLACK team.

# Use GitHub Copilot to autocomplete most of this, except f
or the Pawns...

# Use a dict comprehension to add the black pawns on row 2,
the keys are the square names and the values are the piece
```

```
s.
black_pawns = {
    # Dict comprehension here
}
# Update the black pawns to the squares property
self.squares.update(black_pawns)
# Repeat for the white pawns, they are on rows 7
```

`print_board()`

Will print all the rows of the board in a row-first way. This will represent the board in a different way than the default dictionary way. The result should look like this:

```
[BLACK Rook 1, BLACK Knight 1, BLACK Bishop 1, BLACK Queen
1, BLACK King 1, BLACK Bishop 2, BLACK Knight 2, BLACK Rook
2]
[BLACK Pawn 1, BLACK Pawn 2, BLACK Pawn 3, BLACK Pawn 4, BL
ACK Pawn 5, BLACK Pawn 6, BLACK Pawn 7, BLACK Pawn 8]
[None, None, None, None, None, None, None, None]
[None, None, None, None, None, None, None, None]
[None, None, None, None, None, None, None, None]
[None, None, None, None, None, None, None, None]
[WHITE Pawn 1, WHITE Pawn 2, WHITE Pawn 3, WHITE Pawn 4, WH
ITE Pawn 5, WHITE Pawn 6, WHITE Pawn 7, WHITE Pawn 8]
[WHITE Rook 1, WHITE Knight 1, WHITE Bishop 1, WHITE Queen
1, WHITE King 1, WHITE Bishop 2, WHITE Knight 2, WHITE Rook
2]
```

You can achieve this by looping over the 8 rows, starting with row `1` if the key name ends with `1`. If you nest this in a list, using the **list comprehensions** syntax, you can loop over them later and print everything.

Git checkpoint

When `Board` setup and printing works, commit it as a separate milestone:

```
git add chess/board.py chess/main.py
git commit -m "feat: add board setup and board printing"
```

`find_piece(symbol: str, identifier: int, color: str)`

(I use Python type hints so the IDE's know what types you expect in the functions) Use a **list** comprehension to loop over all the `self.squares.items()` and return those where the `symbol`, `identifier` and `color` are the same **and** the value **is not None**

Other scripts

Use these scripts as well, they might come in handy later

```
def get_piece(self, square):
    """Returns the piece that is on a specific square"""
    return self.squares[square]

def is_square_empty(self, square):
    """Returns True if the square is empty, False otherwis
e."""
    return self.get_piece(square) is None
```

`kill_piece(square)`

This method should be called to kill a certain piece on a certain square. You can call the `die()` method of the piece you're searching for.

Implementing the movements on the board

Now that we have our board setup, we can start programming the BoardMovements. Again, let's learn how to work with Classes, **StaticMethods** and explore the benefits of **GitHub Copilot** to do our repeated work again.

Define a BoardMovement class

We want to define a class called `BoardMovement` that does not inherit from anything and will not be implemented by other classes. This class will implement a few `@staticmethods` that we will use to update the position of the ChessPieces.

Forward

To show how a piece can move forward, let's implement it by updating our rows.

```
@staticmethod
def forward(position: str):
    """
    Move the piece forward on the board.
    """
    # Get the column and row of the current position
    column = position[0] # First character defines the colu
mnn
    row = int(position[1]) # The second character is a nume
rical representation and defines the row.

    # A forward movement increments the rows
    new_row = row + 1

    # Return the new position
    return f"{column}{new_row}"
```

Backward, Left, Right

This function will work, and will just update the new row. Using this same logic, we can create a `backward` functionality, and a `left` and `right` functionality.

More dynamic movements

Extend the functionality so that you can give a dynamic amount of squares, for the `Rook` can travel for more than just one square.

During testing, you will notice that the `WHITE` pawns cannot move forward anymore, as they should travel "backwards" instead (the board is flipped 180 degrees for the opponent players). Make sure the `color` property toggles the direction of `forward` and `backward`.

When you reach the edges of the board, the movement should be blocked. For the rows this means the `new_row` cannot be `0` or `9`. The columns are a little trickier, because subtracting `1` from the `ord('a')` leaves you with the '`' character.

```
if (new_column == '`' or new_column == 'i'):
    # ...
```

Combination movements (Diagonally)

The Bishop, Queen and King can all move diagonally. Make sure it's possible for them to do so. You can combine the functionalities of the `Right` and `Forward` methods, to combine a diagonally forward movement, for example. Create it for all the possible combinations (4 directions (forward-left, forward-right, backward-left, backward-right))

Knight movements

The Knight movements are a little more tricky. It's a different approach to the diagonal movements, as they have an `L`-shaped movement. This means that `forward-left` means that it goes `2` squares forward and `1` square left. The `left-forward` does the opposite. It goes `1` square forward and `2` squares left. Program this specifically for the Knights, it does not have to be added into the BoardMovements class.

Using the movements

Now that you have defined the movements for all the different possible movements, we can start using them into the `move()` methods in the base class `BaseChessPiece` as well as in the **Child** classes.

For the `Pawn`, we define it's `move()` method like this:

```
def move(self):
    movement = BoardMovements.forward(self.position, self.c
olor, 1)
    super().move(movement)
```

For some other pieces, it's needed to dynamically pass the direction and the amount of squares.

```
class Rook(BaseChessPiece):

    def move(self, direction, squares):
        if (direction == 'Left'):
```

```
            movement = BoardMovements.left(self.position, s
elf.color, squares)
        # ...
        super().move(movement)
```

Implementing the movement onto the board

To implement the movement onto the board, we need to add some functionality to the `BaseChessPiece.move()` function. We already passed our `movement` parameter to this method, so we can start working with it now. Due to the implementation with the `BoardMovements` our `movement` variable will contain the new square a certain piece should move to. This essentially means that we should move the object in the `Board.squares` dictionary from one key to the other. We could do that by manually setting the **old_position** to `None` and placing the ChessPiece into a new dictionary key.

We do however encounter one problem with this setup. There is currently no way for the ChessPieces to access anything related to the Board. The ChessPieces also don't have any information about their current position, as it's defined by the key they are in: `{"a1": "BLACK Rook 1", ...}`

A way to solve this, is to add a `board` property to the `BaseChessPiece`. This one will be filled in by the `Board` class after their init has succeeded.

We will immediately fill in the `position` property now as well. As this happens in the same step of the code.

```
# board.py
class Board():
    def __init__(self):
        # define the squares using your dict comprehension
        self.setup_board()

        # loop over the squares as keys and pieces as value
s, in one statement you fetch them using the `dict.items()`
option.
        for square, piece in self.squares.items():
            # Check if piece is not None:
            if piece is not None:
```

```
                    piece.set_initial_position(square)
                    piece.define_board(self)
```

These two lines call two methods that are yet to be defined into our `BaseChessPiece` class.

As there is nothing special about it, I can give them to you here. If you want to test it yourself, don't click open the spoiler yet ;-) That way you can test your Python knowledge some more!

▼ **Solutions to the other methods of the BaseChessPiece**

```
def set_initial_position(self, position):
    self.position = position

def define_board(self, board):
    self.board = board
```

Implementing the movement onto the board, Take 2!

Now that we have all the information we need, we can finally start adding our movements into the Base `move()` method.

I'll give you the first batch of code, which you can then adapt to customise towards your need by filling in the code using the comments.

```
def move(self, movement):
    # Get the new location to see if there is space
    new_location = self.board.get_piece(movement)

    ## TODO: Check to see if new_location is free.
    ## If it's an enemy, kill it instead!
    ## Note: I know that this doesn't work for Pawn killing
s, but that's beside the point for this demonstration proje
ct...

    ## Note 2: I also know that you can't jump over

    # Remove yourself from the old position
```

```
    self.board.squares[self.position] = None
    # Reposition
    self.position = movement
    # Move yourself to the new position
    self.board.squares[self.position] = self
```

You can test these functionalities in the `main.py` script where you can move any Piece and print out the board after each movement.

...

Do you notice anything?

Printing the board after each movement is quite the hassle, right?

Why don't we automate this by adding a Decorator on top of the function so that it automatically logs and prints the movement?

When working with the decorators in the BaseChessPiece, we need to import the `functools` library where you can create a decorator from a private class method.

The structure is like this:

```
def print_board(func):
    @functools.wraps(func)
    def wrapper(self, *args, **kwargs):
        # Actual code
    return wrapper
```

You can now add this decorator over the `move()` function.

```
@print_board
def move(self, movement):
    # ...
```

Git branching tip

For larger experiments (decorators, generators, or rule validation), create a feature branch first:

```
git checkout -b feature/decorators-and-board-state
```

Merge back into `main` only after your tests or manual checks pass.

Saving the board states

It will be useful to save the board states to a text file so the whole gameplay can be replayed and visualised later on.

We can do so using this small code snippet:

```
with open('board.txt', 'a') as file:
    file.write(json.dumps(self.squares))
```

Place this in the `Board` class in a new function (choose the name yourself).

> 💡 Using this `json.dumps` requires our BaseChessPiece objects to be serialised into JSON format in order to represent it in a string format. You could define this in a easy **pythonic** way by making the `BaseChessPiece` inherit the `dict` class: `class BaseChessPiece(ABC, dict)`. In the constructor, you can now set the dict values using `dict.__init__(self, color=color, symbol=symbol, ...)` (Whatever you want to save)

As we also want to save the board after every move, use the same concept as we did with the decorator for the `print_board`, but this time we use `save_board`. Use the `self.board` to access the board after every movement.

Loading in the states of the board using a generator

As we have saved our board to a txt file, we can reuse this file to read in the states. However, after a long time, the amount of memory this takes when reading all the lines at once can become quite big. We can solve this by using a generator which will only read one line at a time. You can then loop over the generator as many times as you want to read all the lines of the files if requested.

As we don't need to get the Board object to be loaded in before we fetch the board states, you can use a `@staticmethod` for this. Create a Static Method that uses the Generator concept to read in the `board.txt` file and return only one line at a time.

If you want to print it in the same format as the current `print_board()` option, you will need to create a seperate `@staticmethod` that uses this dictionary way of saving the object.

Experiment with the rest of these methods as much as you like!

Expanding on the functionalities

I am well aware that not all the functionalities of the Chess game have been implemented. When you want to explore some more Python features, please go ahead and try to create and program more functionalities.

Examples:

- Killing Pawns diagonally (They move vertically, but kill diagonally)
- Block the movement if another piece is already there
- Block the movement if another piece is in the way (Which is the case for all the movements except the Knight, they can jump over other pieces)
- Create a turn system where White and Black cannot perform two turns after eachother.
- Make the game interactive by using IPython or Jupyter Notebooks and inputting the new movements instead of programming them all in one `main.py` file.
- Implement `en passant`, `castling` (NL: `rokade`) and `promotion` for the relevant pieces.
- Program `Check`, `Checkmate` and `Stalemate` into the game.
- Define a `WIN` or `LOSE` calculation.

Final Git checklist

- Run `git status` and ensure only intended files changed.
- Use clear commit messages (`feat:`, `fix:`, `refactor:`, `docs:`).
- Push your latest branch: `git push`.
- Open a Pull Request on GitHub and describe what you implemented

Pull Request checklist (recommended)

Before opening your PR:

- Rebase or merge latest `main` into your feature branch.
- Ensure there are no conflict markers left in files.
- Verify your app still runs (`main.py`) and board output is correct.

Open your PR with:

- Title: `feat: add board setup and piece movement`
- Description:
  - What changed
  - Why you changed it
  - How to test it
  - Optional screenshots/output snippets

Helpful PR flow commands:

```
git checkout feature/board-setup
git fetch origin
git merge origin/main
git push
```

▼ Some helpful code repos to check out!

https://github.com/nathansegers/backend-at-home-01